# Agentic Project Management

BSc Thesis Case Study

## Participation Guide



Konstantinos Chaidemenos

National and Kapodistrian University of Athens

Department of Informatics and Telecommunications

# 1. Welcome

Thank you for taking part in this research. The study compares two AI-assisted software development frameworks, Agentic Project Management (APM) and GitHub Spec-kit, on a fixed coding task. Your contribution helps build the empirical evidence base for the thesis.

You are not being evaluated. The frameworks are. Whatever you produce during your sessions, including incomplete or non-functional code, is valuable data.

## 1.1. Data privacy and anonymity

Submissions are handled with strict confidentiality. No personal identifier is linked to your data. You have been assigned a participant ID (e.g., `P001`); use it in your submission filenames as described later. AI interaction logs are encrypted with a key held only by the research team before being added to the published research dataset. The dataset's structure is transparent; the contents of your sessions remain private.

By proceeding, you consent to the anonymous collection and use of this data as described.

## 1.2. Before you start

You have been provided separately with:

- your participant ID;
- your per-session framework assignment (which framework to use in session 1 and which in session 2);
- credentials for the Anthropic Claude Max subscription you will use during the sessions.

Keep these to hand. They do not appear anywhere in this guide or in the participant package.

Read the official documentation of your assigned framework before each session begins; a short throwaway run on an unrelated repository is recommended for first-time familiarity.

- Agentic Project Management (APM): https://agentic-project-management.dev.
- GitHub Spec-kit: https://github.github.io/spec-kit/.

# 2. Study overview

## 2.1. What you do

You will complete the same coding task twice, in two separate three-hour sessions, each using a different framework. The two sessions can run back to back, on the same day, or on different days. They must not overlap, and you must approach the second session as if the first had not happened, working from a fresh checkout of the codebase.

### 2.2. Time

Each session is strictly time-boxed to three hours. Stop at the three-hour mark whether or not the task is complete. Submitting unfinished work is an expected and valid outcome.

### 2.3. What you submit

After each session you produce one zip file containing two artefacts at its root:

- `solution.patch`: a unified diff of your work against a pinned starting commit of the implementation codebase.
- `transcripts/`: a flat directory of the AI interaction logs (one `.jsonl` file per conversation).

The zip filename follows the strict pattern `{PID}_S{1|2}_{apm|speckit}.zip`, e.g., `P001_S1_apm.zip`.

You also answer a short post-session questionnaire about your experience with the framework, submitted on the same form alongside the zip.

## 3. Participant package

This guide ships inside a participant-package zip alongside the task specification and the helper scripts. Its layout:

```
participant-package/
|-- participant-guide-cc.pdf       # this document
|-- task/
|   |-- PRD.md                     # Product Requirements Document
|   |-- PROMPT.md                  # default opening prompt for the AI
|   `-- README.txt                 # one-shot setup notes; delete after use
|-- scripts/
|   |-- list-chats.py              # transcript listing helper
|   |-- collect-chats.py           # transcript collection helper
|   `-- reset-chats.py             # transcript wipe helper
`-- .claude/
    `-- skills/
        `-- bsc-apm-study-helper/  # optional Claude Code skill (see Section 3.1)
            |-- SKILL.md
            `-- references/...
```

`PRD.md` is the sole source of truth for what to implement. The three Python scripts under `scripts/` manage the AI's transcript files; you run them yourself between or after sessions, or you have a Claude Code session with the optional helper skill below loaded run them on your

behalf with confirmation. They accept your workspace path as an argument so they can sit anywhere on disk.

### 3.1. Optional helper skill

The package ships an optional Claude Code skill, pre-extracted at `.claude/skills/bsc-apm-study-helper/`, that can answer logistics questions about this guide: setup, the helper scripts, packaging, the between-sessions reset, cleanup. The skill is project-scoped: Claude Code auto-discovers it only when you open Claude Code from the participant-package directory itself. To use it, open a Claude Code session in the participant-package directory:

```
cd path/to/participant-package
claude
```

You then ask logistics questions in that session. To stop using the skill, simply close that session.

**Note.** The skill is intentionally project-scoped to the participant-package directory, not user-level. It is therefore not loaded in your task's workspace, only in a Claude Code session opened from the participant-package directory. Do not copy it to `~/.claude/skills/` or to your workspace's `.claude/skills/`; doing so would defeat that isolation.

**Warning.** The skill is for logistics only. It is instructed to refuse questions about the implementation task or the frameworks themselves and to redirect you to their documentation, but the agent may still slip; if it does, the responsibility to disregard such answers lies with you. Acting on, paraphrasing, or carrying any task or framework content the helper produced into a session contaminates your study data.

## 4. System requirements

### 4.1. Operating system

A Linux environment is required. Two paths are supported:

- **Native Linux** (Ubuntu 22.04 or 24.04, or an equivalent distribution).
- **Linux virtual machine** on a non-Linux host. The virtual machine must satisfy every other requirement in this section, and the work happens entirely inside it: the participant package, the codebase clones, the workspace, and the mock environment all live on the VM filesystem.

**macOS.** Two Homebrew-installable VM tools are recommended, split by host architecture. Both are free, open source, CLI-driven, and bring up an Ubuntu LTS VM in one command. Homebrew (`https://brew.sh`) must be present first.

Apple Silicon Macs use Multipass:

```
brew install --cask multipass
multipass launch lts --name task --cpus 4 --memory 8G --disk 20G
multipass shell task
```

Intel Macs use Lima:

```
brew install lima
limactl start --name=task --cpus=4 --memory=8 --disk=20 \
    --tty=false template:ubuntu-24.04
limactl shell task
```

On macOS, if neither Multipass nor Lima works on your host, UTM (https://mac.getutm.app) is a secondary option: free, open source, GUI-driven. Install Ubuntu 24.04 from the official ISO; setup takes longer than the CLI tools because the Ubuntu installer runs interactively.

**Windows.** Install WSL2 with an Ubuntu 24.04 distribution following the official Microsoft instructions at https://learn.microsoft.com/en-us/windows/wsl/install, then follow the native-Linux path inside the WSL2 shell.

> **Warning.** Docker Desktop is not a substitute. The mock environment described in Section 7 runs Docker Engine *inside* the VM, not on the host.

### 4.2. System resources

The Linux environment needs at least 4 vCPUs, 8 GB of RAM, and a 20 GB virtual disk; the launch commands above provision these for VM users. After Docker images are loaded (about 1.6 GB for the openeclass container), at least 5 GB of free disk should remain for the workspace, Docker state, and transient files.

### 4.3. Tools and libraries

The case-study work needs a container runtime (Docker Engine with Compose v2), a C build chain (`gcc`, `make`, `valgrind`, `pkg-config`, plus the `libcurl` and `libxml2` development headers), two scripting runtimes (Python 3.10 or newer with `uv`, and Node.js with `npm`), and a handful of command-line utilities (`git`, `curl`, `tar`, `openssl`, `bash` 3.2 or newer, and `gh`). Install Docker first via the official convenience script:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

The new `docker` group membership takes effect on your next login or after `newgrp docker`. Install everything else with `apt` and the official `uv` installer:

```
sudo apt-get update
sudo apt-get install -y \
    build-essential pkg-config valgrind \
    libcurl4-openssl-dev libxml2-dev \
    git curl tar openssl gh python3 python3-pip npm
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If `uv -version` reports `command not found` immediately after the install, run `source ~/.local/bin/env` once or open a new login shell so the installer's PATH update takes effect. The C minimums are `gcc` 9.0 (for C11), `libcurl` 7.81.0, and `libxml2` 2.9.13; Ubuntu 22.04 and 24.04 both satisfy them. Verify the toolchain:

```
gcc --version
pkg-config --modversion libcurl
pkg-config --modversion libxml-2.0
valgrind --version
python3 --version
node --version
npm --version
uv --version
```

## 5. AI tools

The session uses a coding-agent CLI (Claude Code) and one of two AI-assisted development frameworks (APM or Spec-kit). The participant package does not bundle any of them. Install Claude Code once before either session, and install the assigned framework's CLI before that session begins. The system requirements in Section 4 cover the underlying tools all three depend on.

### 5.1. Claude Code

Install the Claude Code CLI by following the official instructions at https://docs.claude.com/en/docs/claude-code. Sign in with the credentials you were provided.

### 5.2. Agentic Project Management (APM)

APM ships as an npm package; the CLI is invoked as `apm`.

```
sudo npm install -g agentic-pm
apm --version
```

### 5.3. GitHub Spec-kit

Spec-kit runs through `uvx` from a release tag of the upstream repository; the CLI is invoked as `specify`, prefixed by the same `uvx --from` expression on every invocation. `v0.8.3` is the floor verified for this study; any newer release tag listed at https://github.com/github/spec-kit/releases is also acceptable.

```
uvx --from git+https://github.com/github/spec-kit.git@v0.8.3 \
    specify --help
```

## 6. Workspace

The participant package's `task/` directory holds `PRD.md` (the requirements you implement against), `PROMPT.md` (the message you give the AI to open the session), and a throwaway `README.txt` with the clone commands for the two repositories at their pinned commits. The *workspace* is the directory in which you open Claude Code on your Linux environment (native or inside the VM); it is the AI's working directory for the session, and it must end up with this exact layout:

```
<workspace>/
|-- PRD.md
|-- PROMPT.md
|-- eclass-mcp-server/      # implementation codebase
`-- openeclass/             # legacy PHP codebase (read-only reference)
```

Create the workspace as a fresh directory of your choice on the Linux filesystem (`~/workspace/` is a reasonable default), copy `PRD.md` and `PROMPT.md` into it from the participant package's `task/` directory, and clone the two repositories alongside them following the next subsection. `README.txt` carries the same clone commands for convenience and has no further role.

### 6.1. Cloning the codebases

Clone `eclass-mcp-server` and check out the pinned commit `dbd2d16`. This commit is the patch baseline: every diff you submit is computed against it.

```
cd <workspace>
git clone https://github.com/sdi2200262/eclass-mcp-server.git
cd eclass-mcp-server
git checkout dbd2d16
uv sync --dev --all-extras
cd ..
```

Clone `openeclass` at its pinned commit `e8b3329`. The AI treats this codebase as a read-only reference; you should not modify it.

```
cd <workspace>
git clone https://github.com/gunet/openeclass.git
cd openeclass
git checkout e8b3329
cd ..
```

# 7. Mock environment

The task integrates with openeclass through a single-sign-on flow, both for the existing Python MCP (Model Context Protocol) server it expands and for the C replica it asks for. The study ships a self-contained *mock environment* that you run locally during each session: a real openeclass instance paired with a mock SSO service whose flow matches `eclass-mcp-server`'s integration, pre-loaded with a fixed test data set so every participant's session runs against identical state.

## 7.1. Download and install

The mock environment ships from the `sdi2200262/bsc-apm-case-study-env` GitHub repository as two architecture-specific releases, each with a single tarball asset:

- Tag `bsc-apm-case-study-env` ships `bsc-apm-amd64.tar.gz` for x86_64 hosts (most Intel and AMD Linux machines, and the Linux VM on an Intel Mac).
- Tag `bsc-apm-case-study-env-arm64` ships `bsc-apm-arm64.tar.gz` for arm64 hosts (Apple Silicon Macs running a Linux VM, arm64 Linux servers, Raspberry Pi 4/5, etc.).

Identify your architecture with `uname -m`: `x86_64` means amd64, `aarch64` or `arm64` means arm64. Pick a prefix on disk for the mock environment, separate from your workspace (the rest of this guide writes `<mock>/` for that path; `~/.bsc-apm` is a reasonable default). Run *one* of the two download commands below, the one that matches your architecture:

```
mkdir -p <mock> && cd <mock>

# x86_64 hosts:
gh release download bsc-apm-case-study-env \
    -R sdi2200262/bsc-apm-case-study-env \
    -p 'bsc-apm-amd64.tar.gz'

# arm64 hosts:
gh release download bsc-apm-case-study-env-arm64 \
    -R sdi2200262/bsc-apm-case-study-env \
    -p 'bsc-apm-arm64.tar.gz'
```

Then unpack and bring the stack up. `./install` loads the bundled Docker images, generates self-signed TLS certificates into `./certs/`, and writes a pre-configured `.env` alongside.

`./scripts/up` brings the three containers up, performs openeclass's first-time install on first run, and applies the test data; this takes a few minutes on a clean host.

```
tar -xzf bsc-apm-*.tar.gz
./install
./scripts/up
```

The stack listens on two URLs once it is ready:

- `http://localhost/` for the openeclass instance.
- `https://localhost:18443/` for the mock authentication service.

Ports 80 and 18443 must be free on the Linux environment. Port 80 is the default HTTP port and is more commonly held by another service than 18443 is; if either is bound, stop the holder before running `./scripts/up`. The helper skill walks through identifying and stopping the holder if needed.

### 7.2. Lifecycle commands

Once installed, manage the environment from its prefix:

- `./scripts/up`: bring the stack up. Idempotent.
- `./scripts/down`: stop the stack; named volumes survive.
- `./scripts/status`: probe each service from the mock side and report whether the stack itself is healthy.
- `./scripts/verify-mcp --mcp-root <path>`: consumer-side complement to `status`; verifies that an `eclass-mcp-server` checkout authenticates against the mock with the `.env` and `certs/` it has been given. Runs only against the pinned baseline (Section 6.1), at initial setup or after a between-sessions reset.
- `./scripts/logs`: tail container logs.
- `./scripts/reset`: drop database state and re-apply the test data.

You should not need to reset during a session. If you do (for instance, after a destructive experiment from the AI), `./scripts/reset` returns the environment to its initial state.

### 7.3. MCP server configuration

The `.env` and `certs/` that `./install` wrote are everything the implementation needs to authenticate against the mock. Copy both into the `eclass-mcp-server/` checkout, preserving the `certs/` subdirectory:

```
cp <mock>/.env <workspace>/eclass-mcp-server/.env
cp -r <mock>/certs <workspace>/eclass-mcp-server/certs
```

The `.env` references the certificate by relative path, so no further configuration is required.

### 7.4. Uninstall

To remove the mock environment from your machine after the study, stop the stack and drop its volumes (`down -v`), remove the loaded images, and delete the prefix:

```
docker compose -f <mock>/compose.yaml down -v
docker rmi bsc-apm/openeclass:dev bsc-apm/mock-cas:dev
rm -rf <mock>
```

## 8. Session workflow

A session has three phases: starting, working, and wrapping up. Between your two sessions, run the reset at the end of this section to return the workspace and the transcript store to the same state the setup steps above produced for session 1.

### 8.1. Starting a session

Note the wall-clock start time, then open Claude Code with the workspace as its working directory. Each framework has its own way of starting; follow your assigned framework's documentation and point it at `PROMPT.md` as the task.

### 8.2. Working on the task

Work for up to three hours and stop at the three-hour mark whether or not you have finished. Use only the framework assigned to your current session and follow its documented workflow as written; the study compares frameworks as documented, not as adapted on the fly. Do not run the participant package's helper scripts during the session, and do not delete any `.jsonl` file from your workspace's transcript directory; those are your transcripts and are needed in your submission.

### 8.3. Wrapping up

**1. Create the patch.** From inside the implementation codebase, stage every change (including new files) and diff against the pinned commit. The `--cached` flag is what pulls newly created source files into the diff, even when the working tree is otherwise clean.

```
cd <workspace>/eclass-mcp-server
git add -A
git diff --cached dbd2d16 > ../solution.patch
```

**2. Collect the transcripts.** Claude Code stores transcripts under `~/.claude/projects/` in a per-workspace subdirectory whose name is the workspace's absolute path with every `/` replaced by `-`. List what is in your workspace's transcript directory and pick the rows that belong to the

session you just finished; the `--from-projects` flag tells `list-chats.py` to resolve the mapping for you:

```
python3 /path/to/scripts/list-chats.py \
    <workspace> --from-projects
```

The script prints one row per transcript with first and last record timestamps and a preview of the first user message. Copy the matching rows out with `collect-chats.py`, pasting the timestamps verbatim into `--from` and `--to` (slightly widen the window to be inclusive):

```
python3 /path/to/scripts/collect-chats.py <workspace> \
    --from 2026-05-01T09:00 --to 2026-05-01T12:30
```

The script copies the matching transcripts into `<workspace>/transcripts/`. Verify the contents of that directory by re-running `list-chats.py` against it with `--from-dir`, which scans the directory directly instead of resolving a workspace path:

```
python3 /path/to/scripts/list-chats.py \
    <workspace>/transcripts --from-dir
```

The output should match the rows you intended to collect, with the same first and last timestamps. If a row is missing, widen the time window and re-run `collect-chats.py`.

**3. Package the submission.** Zip `solution.patch` and `transcripts/` together at the workspace root. The filename uses your participant ID, the session number, and the framework you used in this session:

```
cd <workspace>
zip -r P001_S1_apm.zip solution.patch transcripts/
```

The zip must contain exactly two entries at its root: `solution.patch` and `transcripts/`. Files under `transcripts/` are flat `.jsonl` files; do not nest them under any subdirectory.

**4. Move the zip out of the workspace.** The between-sessions reset wipes everything in the workspace, including the zip. Store it in a directory outside the workspace and outside the participant package; `~/Documents/bsc-apm-submissions/` is a reasonable default. Both session zips end up there, and the cleanup steps later leave that directory untouched.

```
mkdir -p ~/Documents/bsc-apm-submissions
mv <workspace>/P001_S1_apm.zip ~/Documents/bsc-apm-submissions/
```

**5. Submit the form.** Open the submission form, fill in the per-session questions, and attach the zip you just stored. Do this immediately after wrapping up, while the session is fresh.

https://forms.gle/yNDWvsBHAHwq8Dks7

### 8.4. Resetting between sessions

Run this after the first session has wrapped up and before the second session starts. Before running any of the steps below, confirm that session 1's submission zip is in your safe-storage directory (e.g., `ls -la ~/Documents/bsc-apm-submissions/` should list `P001_S1_*.zip`); the reset is destructive and only zips outside the workspace survive. The reset then returns the workspace and the transcript store to the same state the setup steps above produced for session 1, so session 2 begins from an identical baseline. The mock environment is left alone unless you have a reason to believe it has drifted from its initial state.

**1. Reset the implementation codebase.** From inside `eclass-mcp-server`, hard-reset to the pinned commit, wipe untracked and ignored files, and rebuild the virtual environment from scratch:

```
cd <workspace>/eclass-mcp-server
git reset --hard dbd2d16
git clean -fdx
uv sync --dev --all-extras
```

`git clean -fdx` removes every untracked file, including ignored files such as `.venv/` and any `__pycache__/` directories. Re-copy `.env` and `certs/` from the mock-environment prefix afterwards (Section 7.3); they are removed by `git clean`.

**2. Reset the reference codebase.** Do the same for `openeclass`, in case the AI inadvertently modified it:

```
cd <workspace>/openeclass
git reset --hard e8b3329
git clean -fdx
```

**3. Sweep the workspace.** Confirm the workspace root contains exactly four entries: `PRD.md`, `PROMPT.md`, `eclass-mcp-server/`, and `openeclass/`.

```
ls -la <workspace>
```

Delete any extra files or directories the AI may have created at the workspace root. If `PRD.md` or `PROMPT.md` were modified during the session, restore them from the participant package's `task/` directory; session 2 must read them in their original form.

**4. Wipe the transcripts.** Clear the transcript store for this workspace; the script touches only that workspace's transcripts:

```
python3 /path/to/scripts/reset-chats.py <workspace>
```

The script lists the files it will delete and asks for confirmation. Run it only after you have safely stored the session's zip; the deletion is permanent.

**5. Reset the mock environment, only if needed.** The mock environment retains its initial state across sessions and usually does not need a reset. Reset it only if a previous session messed with the seeded data:

```
cd <mock>
./scripts/reset
```

**6. Confirm the baseline.** After the reset, confirm the workspace and the transcript store match the same state setup produced for session 1:

- `<mock>/scripts/status` reports every service ok.
- `<workspace>/eclass-mcp-server` is at `dbd2d16` with a clean working tree.
- `<workspace>/openeclass` is at `e8b3329` with a clean working tree.
- The workspace root contains `PRD.md`, `PROMPT.md`, `eclass-mcp-server/`, and `openeclass/`, and nothing else; `PRD.md` and `PROMPT.md` match their original contents from the participant package.
- Your workspace's transcript directory contains no transcripts from session 1.

Once the baseline is in place, return to Section 8.1 and repeat the same starting-working-wrapping-up flow for session 2 (with your other assigned framework). Session 2 ends after Section 8.3, step 4 (*Move the zip out of the workspace*); the resetting steps are not needed to be run again.

### 8.5. Cleaning up

Optional. Once both sessions are wrapped up and submitted, nothing in this study needs to stay on your machine. If you want a clean slate:

- Wipe session 2's transcripts: `python3 /path/to/scripts/reset-chats.py <workspace>`.
- Delete the workspace: `rm -rf <workspace>`.
- Uninstall the mock environment: follow the steps in Section 7.4.

The participant package itself can be deleted at this point as well; it has no further role.

## 9. Contact

This study is conducted for a Bachelor's thesis by Konstantinos Chaidemenos (CobuterMan).

`sdi2200262@di.uoa.gr` Email
`cobuter_man` Discord

Thank you for your time and contribution to this research.